

Relazione Generale di Progetto — Building AI Agent

Introduzione

Building AI Agent è un progetto didattico progressivo per costruire un agente AI da zero. Il progetto parte da un agente molto piccolo basato su notebook ed evolve passo dopo passo attraverso routing, parsing, esecuzione, memoria, metadati dei tool, integrazione di modelli linguistici locali e infine esecuzione multi-step.

Il progetto non parte da un framework e non parte da un'API esterna di inferenza. Introduce invece gradualmente ogni responsabilità architetturale, in modo che ogni scelta di design possa essere compresa con chiarezza.

Obiettivo del progetto

L'obiettivo del progetto è costruire un agente capace di:

- ricevere una richiesta dell'utente;
- comprendere l'intento della richiesta;
- selezionare il tool corretto;
- estrarre gli argomenti richiesti;
- validare ed eseguire il tool selezionato;
- salvare lo stato dell'interazione;
- gestire piccole richieste multi-step;
- evolvere attraverso un'architettura modulare chiara.

Filosofia di sviluppo

Il progetto segue una regola molto semplice:

one version, one concept

Ogni versione introduce una sola idea architetturale importante. Questo rende il percorso di apprendimento controllato e l'evoluzione facile da seguire.

```
simple agent
↓
separated execution
↓
generic parsing
↓
planning
↓
managed memory
↓
tool registry
↓
LLM routing
↓
LLM parsing
↓
multi-step execution
```

Architettura generale

Nel corso del progetto, l'agente è progettato come una pipeline di componenti specializzati. Nella versione finale implementata il flusso semplificato è:

```
User Request
↓
Planner
↓
LLM Router
↓
LLM Parser
↓
Executor
↓
Memory Manager
↓
Memory / Output
```

Il Tool Registry supporta questo flusso memorizzando funzioni, parametri, descrizioni e sinonimi dei tool. Il Registry è condiviso da Router, Parser, Planner ed Executor.

Componenti principali

Router

Il Router decide quale tool deve essere usato. Nelle prime versioni è rule-based; nella Versione 7 diventa un LLM Router che usa un piccolo modello locale e le descrizioni dei tool memorizzate nel Tool Registry.

Parser

Il Parser converte il linguaggio naturale in argomenti strutturati. Nella Versione 3 diventa generico e riutilizzabile. Nella Versione 8 diventa un LLM Parser, capace di estrarre semanticamente valori da richieste più naturali.

Executor

L'Executor valida gli argomenti ed esegue la funzione selezionata. Esiste per mantenere la logica di esecuzione separata dall'Agent coordinatore.

Planner

Il Planner crea un piano di esecuzione. Nella Versione 4 introduce una pianificazione esplicita prima dell'esecuzione. Nella Versione 9 evolve in un semplice planner multi-step capace di produrre una lista ordinata di step.

Memory Manager

La memoria salva lo stato di ogni interazione. Nella Versione 5 la gestione della memoria viene estratta in funzioni dedicate. Nelle versioni successive la memoria conserva l'intero stato multi-step, compresi piani e step eseguiti.

Tool Registry

Il Tool Registry viene introdotto nella Versione 6 per centralizzare i metadati dei tool. Memorizza funzione, parametri, descrizione e sinonimi di ogni tool e diventa la fonte condivisa di verità per il resto del sistema.

Roadmap implementata

V1 → Agent Base
V2 → Executor
V3 → Generic Parser
V4 → Planner
V5 → Memory Manager
V6 → Tool Registry
V7 → LLM Router
V8 → LLM Parser
V9 → Multi-step Agent

La Versione 9 è la versione finale implementata dell'attuale roadmap di progetto.

Sintesi delle versioni

V1 — Agent Base

Introduce la struttura minima dell'agente con routing, parsing, tool e uno stato semplice.

V2 — Executor

Separa validazione ed esecuzione dall'Agent, in modo che l'Agent diventi un coordinatore.

V3 — Generic Parser

Sostituisce la logica di parsing rigida con un componente riutilizzabile capace di estrarre argomenti per più tool.

V4 — Planner

Aggiunge un piano esplicito che spiega quale tool deve essere usato, perché e quali argomenti sono necessari.

V5 — Memory Manager

Introduce funzioni dedicate per la memoria come salvataggio, lettura, pulizia e recupero dell'ultima interazione.

V6 — Tool Registry

Centralizza i metadati dei tool ed elimina la necessità di strutture separate sparse nel notebook.

V7 — LLM Router

Introduce la selezione semantica del tool tramite un modello linguistico locale, mantenendo stabile il resto dell'architettura.

V8 — LLM Parser

Introduce l'estrazione semantica degli argomenti tramite lo stesso modello linguistico locale e restituisce argomenti strutturati in stile JSON.

V9 — Multi-Step Agent

Introduce l'esecuzione sequenziale. Il planner crea più step, l'Agent li esegue uno alla volta e il risultato di uno step può essere riutilizzato nel successivo.

Valore didattico

Il progetto insegna più del semplice coding. Insegna a ragionare sull'architettura di un agente attraverso separazione delle responsabilità, design modulare, sviluppo incrementale, uso controllato di modelli linguistici locali, esecuzione basata su tool, gestione della memoria e pianificazione.

Architettura finale e ambito finale

L'ambito finale implementato del progetto è un agente multi-step basato su notebook. È volutamente semplice e didattico. Il sistema può:

- effettuare routing semantico con un LLM locale;
- estrarre semanticamente gli argomenti con lo stesso LLM locale;
- salvare i metadati dei tool in un Tool Registry;
- creare un piccolo piano di esecuzione;
- eseguire più step ordinati;
- passare il risultato di uno step a quello successivo;
- salvare l'intero stato dell'interazione in memoria.

Per chiarezza vengono mantenuti volutamente alcuni limiti. La logica multi-step dipende da una semplice divisione tramite la parola "then". Il progetto rimane basato su notebook, il registry è in memoria e la memoria è una semplice lista. Queste scelte sono coerenti con lo scopo didattico del lavoro.

Conclusione

Building AI Agent è un progetto di apprendimento strutturato che mostra come un agente AI possa evolvere da un semplice notebook rule-based in un sistema modulare con Tool Registry, componenti basati su modelli locali ed esecuzione multi-step.

La sua forza principale è l'evoluzione controllata. Ogni versione aggiunge un nuovo concetto e migliora l'architettura senza introdurre complessità non necessaria. Il progetto si conclude ora alla Versione 9, in cui il risultato finale implementato è un agente multi-step chiaro e funzionante, costruito interamente in notebook.